

□ 1. What is Django?

□ Definition

Django is a **high-level Python web framework** used to build secure, scalable, and fast web applications.

□ Key Features

- Follows **MVT architecture**
- Built-in **authentication system**
- Uses **ORM (no need SQL)**
- Admin panel available
- Secure (CSRF, SQL injection protection)

□ Interview Answer

Django is a high-level Python web framework used for rapid development of secure and scalable web applications. It follows MVT architecture and provides built-in features like ORM and authentication.

□ 1. What is Authentication System?

□ Simple Understanding

Authentication = **verifying who the user is**

□ Example:

- Login with username & password
 - System checks → valid or not
-

□ Interview Answer

Authentication is the process of verifying the identity of a user, usually through credentials like username and password, to allow access to the system.

□ Why Django has Built-in Authentication?

□ Simple

Django already provides login system → no need to build from scratch

□ Interview Answer

Django provides a built-in authentication system to handle user login, logout, password management, and permissions securely without needing to implement these features from scratch.

☐ **Advantages**

- Secure
- Time-saving
- Ready-to-use
- Handles users & permissions

☐ **2. What is Django Admin Panel?**

☐ **Simple Understanding**

Admin panel = **ready-made dashboard to manage data**

☐ **Example:**

- Add users
 - Delete data
 - Update records
-

☐ **Interview Answer**

Django admin panel is a built-in interface that allows developers to manage database records such as users and data without writing additional code.

☐ **Key Points**

- No frontend needed
- Easy data management
- Automatically generated

☐ **3. What is SQL Injection?**

☐ **Simple Understanding**

SQL Injection = **a hacking technique**

☐ **Hacker tries to:**

- Access database
- Modify data

☐ **Example**

Instead of normal input:

admin

Attacker writes:

admin' OR '1'='1

☐ This can bypass login ☐

☐ **Interview Answer**

SQL injection is a security vulnerability where an attacker inserts malicious SQL queries into input fields to access or manipulate the database.

☐ **4. How Django Prevents SQL Injection?**

☐ **Simple**

Django uses **ORM instead of raw SQL**

☐ **Interview Answer**

Django prevents SQL injection by using ORM, which automatically escapes user input and ensures queries are safe.

☐ **Additional Protection**

- Input validation
 - Parameterized queries
-

☐ **FINAL QUICK ANSWERS (REVISION)**

☐ Authentication

= Verify user identity

□ Django Authentication

= Built-in login system

□ Admin Panel

= Manage database easily

□ SQL Injection

= Attack using malicious queries

□ Protection

= ORM + safe queries

□ HOW TO ANSWER IN INTERVIEW (IMPORTANT)

□ Keep it simple:

Authentication verifies users. Django provides a built-in authentication system to handle login and security. The admin panel is used to manage data easily. SQL injection is a database attack, and Django prevents it using ORM.

□ 2. Virtual Environment

□ Definition

A virtual environment is an **isolated Python environment** for managing project-specific dependencies.

□ Purpose

- Avoid dependency conflicts
- Maintain project isolation
- Keep system clean

□ Steps to Create

Step 1: Create

```
python -m venv env
```

Step 2: Activate

- Windows:

```
env\Scripts\activate
```

- Mac/Linux:

```
source env/bin/activate
```

Step 3: Install Django

pip install django

Step 4: Deactivate

deactivate

☐ Interview Answer

A virtual environment is used to isolate dependencies for each project and avoid conflicts between different versions of packages.

☐ 3. MVT Architecture

☐ Components

- **Model** → Database
- **View** → Business Logic
- **Template** → UI

☐ Flow

User → URL → View → Model → Template → Response

☐ Interview Answer

Django follows MVT architecture where Model handles data, View processes logic, and Template renders the UI.

☐ 4. Models

☐ Definition

Model represents **database structure** using Python classes.

☐ Example

```
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    age = models.IntegerField()
```

☐ Key Points

- Maps to database table
- Each field = column
- Uses ORM

☐ Interview Answer

Model defines the structure of the database using Python classes, where each class represents a table.

□ 5. Views

□ Definition

View handles **request processing and business logic**

□ Example

```
def home(request):  
    return render(request, 'home.html')
```

□ Interview Answer

View processes user requests, interacts with models, and returns a response such as HTML or JSON.

□ 6. Templates

□ Definition

Template is used to **display data (frontend UI)**

□ Example

```
<h1>{{ name }}</h1>
```

□ Interview Answer

Templates are used to render dynamic data using HTML and Django Template Language.

□ 7. ORM (Object Relational Mapping)

□ Definition

ORM allows database interaction using **Python instead of SQL**

□ Examples

```
Student.objects.all()  
Student.objects.filter(age=20)
```

□ Advantages

- No SQL needed
- Easy to use
- Database independent

□ Interview Answer

ORM allows developers to interact with the database using Python code instead of writing SQL queries.

□ **8. URL Routing**

□ **Definition**

URL routing maps **URLs to views**

□ **Example**

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.home)
]
```

□ **Interview Answer**

URL routing maps incoming requests to the appropriate view functions.

□ **9. Request → Response Flow**

□ **Flow**

1. User sends request
2. URL dispatcher matches URL
3. View is called
4. View interacts with Model
5. Data sent to Template
6. Template returns response

□ **Interview Answer**

When a user sends a request, Django routes it through URL dispatcher to a view, which processes it and returns a response via template.

□ **10. REST API Basics**

□ **Definition**

REST API allows communication between frontend and backend using HTTP methods.

□ **HTTP Methods**

- GET → Fetch data
- POST → Send data
- PUT → Update
- DELETE → Remove

□ Data Format

- JSON

□ Interview Answer

REST API is a method of communication between client and server using HTTP methods and JSON data format.

□ 11. API Integration in Django

□ Definition

API integration means **connecting Django with external/internal services**

□ Calling External API

Example

```
import requests
```

```
def get_data(request):  
    response = requests.get("https://api.example.com")  
    data = response.json()  
    return render(request, "home.html", {"data": data})
```

□ Flow

View → API → JSON → Template

□ Interview Answer

We use the requests library to call external APIs, receive JSON data, and use it in the application

□ 12. Creating API using Django REST Framework (DRF)

□ Steps

1. Install DRF

```
pip install djangorestframework
```

2. Add to settings

```
INSTALLED_APPS = ['rest_framework']
```

3. Serializer

```
class StudentSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Student  
        fields = '__all__'
```

4. API View

```
@api_view(['GET'])  
def student_list(request):  
    students = Student.objects.all()  
    serializer = StudentSerializer(students, many=True)  
    return Response(serializer.data)
```

5. URL

```
path('api/students/', views.student_list)
```

☐ Interview Answer

We create APIs in Django using Django REST Framework by defining serializers, views, and URL endpoints.

☐ 1. MIGRATIONS (VERY IMPORTANT)

☐ Definition

Migration is the process of **creating/updating database tables from models**

☐ Commands

```
python manage.py makemigrations  
python manage.py migrate
```

☐ Simple Understanding

- You create model → Django converts it to DB table
- Migration = bridge between model & database

Questions

- What is migration?
- Difference between makemigrations and migrate?

☐ Answer:

- **makemigrations** → creates changes
- **migrate** → applies changes

□ 3. FORMS IN DJANGO

□ Definition

Forms are used to **take user input**

Types

- HTML Forms
 - Django Forms
-

Example

```
from django import forms
```

```
class StudentForm(forms.Form):  
    name = forms.CharField(max_length=100)
```

□ Interview Answer

Django forms are used to handle user input, validation, and processing in a secure way.

□ 4. SERIALIZATION (VERY IMPORTANT FOR API □)

□ Definition

Serialization = **convert complex data** → JSON

□ Simple Understanding

Model → JSON

JSON → Model

Example (DRF)

```
class StudentSerializer(serializers.ModelSerializer):  
    class Meta:
```

```
model = Student
fields = '__all__'
```

☐ Interview Answer

Serialization is the process of converting complex data like Django models into JSON format so it can be sent through APIs.

☐ Definition

Middleware is something that runs **between request and response**

☐ Example

- Authentication check
 - Logging
 - Security
-

☐ Interview Answer

Middleware is a layer that processes requests and responses globally before reaching the view or after leaving it.

☐ Static Files

- CSS, JS, Images

☐ Media Files

- User uploads
-

☐ Interview Answer

Static files are used for frontend assets, while media files are used for user-uploaded content.

☐ “If DB change not reflecting?”

Answer:

I will check if migrations are applied properly.

☐ “How API sends data?”

Answer:

Using serializer to convert data into JSON.

- ☐ manage.py → project manager
- ☐ settings.py → configuration
- ☐ urls.py → routing
- ☐ models.py → database
- ☐ views.py → logic

1. DJANGO PROJECT STRUCTURE (VERY IMPORTANT)

- ☐ **You MUST know this**

Key Files:

- manage.py → project manager
 - settings.py → configuration
 - urls.py → routing
 - models.py → database
 - views.py → logic
-

- ☐ **Interview Answer**

A Django project contains files like settings.py for configuration, urls.py for routing, models.py for database structure, and views.py for business logic.

2. APP vs PROJECT (VERY COMMON QUESTION)

- ☐ **Simple**

- Project = whole application
 - App = module inside project
-

- ☐ **Interview Answer**

A project is the complete application, while an app is a component that handles a specific functionality within the project.

GET	POST
Fetch data	Send data
Data in URL	Data in body

Less secure	More secure
FunctionBased View	Class Based View
Simple	Reusable
Easy	More structured

☐ 5. DATABASE CONNECTION (IMPORTANT)

☐ Where configured?

☐ settings.py

☐ Interview Answer

Database configuration is defined in settings.py where we specify database engine, name, user, and password.

☐ 6. DJANGO LIFECYCLE (VERY IMPORTANT ☐)

☐ Flow

Browser → URL → View → Model → Template → Response

☐ Interview Answer

When a request is sent, Django routes it through URL dispatcher to a view, processes logic, interacts with model, and returns response via template.

☐ 7. CSRF TOKEN (SECURITY)

☐ What is it?

Prevents **unauthorized requests**

☐ Interview Answer

CSRF token is used to protect web applications from **cross-site request forgery** attacks.

8. DEBUG MODE

☐ In settings.py

DEBUG = True

☐ Interview Answer

Debug mode is used during development to display detailed error messages. It should be disabled in production.

☐ 9. DEPLOYMENT BASIC IDEA (BONUS)

☐ Simple

- Run Django on server
 - Use Gunicorn / Nginx
-

☐ Interview Answer

Django applications are deployed using web servers like Nginx and application servers like Gunicorn.

☐ 1. What is Debugging?

☐ Simple Understanding

Debugging = **finding and fixing errors (bugs) in code or system**

☐ Example:

- Website not loading
 - API not working
 - Login failed
-

☐ Interview Answer (Say this ☐)

Debugging is the process of identifying, analyzing, and fixing errors or issues in a program or system to ensure it works correctly.

☐ Real-Time Example (VERY IMPORTANT)

☐ Say like this in interview:

If a user reports that the website is not loading, I will check server status, logs, API responses, and network issues to identify and fix the problem.

☐ **2. Debugging Steps (VERY IMPORTANT ☐)**

☐ **Standard Process**

1. **Understand the issue**
2. **Reproduce the problem**
3. **Check logs/errors**
4. **Identify root cause**
5. **Fix the issue**
6. **Test again**

First I understand the issue, then reproduce it, check logs and errors, identify the root cause, fix it, and test the solution.

3. Debugging in Django

☐ **Common Things to Check**

- Console errors
 - Django logs
 - Database connection
 - API response
 - URL routing
-

☐ **Example Answer**

In Django, I debug issues by checking server logs, verifying URL routing, validating API responses, and ensuring database queries are correct.

☐ **Scenario 1: Website not loading**

☐ **Answer:**

I will check if the server is running, verify URL routing, check logs for errors, and ensure database connection is working.

☐ **Scenario 2: API not working**

☐ **Answer:**

I will check API endpoint, request method, parameters, response status code, and server logs.

☐ **Scenario 3: Login not working**

☐ **Answer:**

I will check user credentials, authentication logic, database records, and session handling.

☐ **Scenario 4: Slow website**

☐ **Answer:**

I will check database queries, API response time, server performance, and network latency.

☐ **5. Tools Used for Debugging**

- Logs (VERY IMPORTANT)
- Browser Developer Tools
- Postman (for API testing)
- Django debug mode

6. Why Debugging is Important?

☐ **Interview Answer**

Debugging helps identify issues quickly and ensures the system works efficiently and reliably.

☐ **1. DJANGO SIGNALS (RARE but BONUS)**

☐ **Definition**

Signals allow actions to happen automatically

☐ **Example:**

- Send email after user signup
-

☐ **2. CACHE (ADVANCED – OPTIONAL)**

☐ **Definition**

Store data temporarily to improve performance

□ 3. PAGINATION

□ Definition

Split large data into pages

□ 4. ERROR HANDLING

□ Example:

try:

```
    data = Student.objects.get(id=1)
except Student.DoesNotExist:
    print("Not found")
```

□ 5. HTTP STATUS CODES (IMPORTANT)

- 200 → Success
 - 404 → Not found
 - 500 → Server error
-

□ 6. SESSION & COOKIES (BASIC IDEA)

- Session → server side
- Cookies → browser

□ What is settings.py?

- Main configuration file

Contains:

- Installed apps
- Database config
- Middleware
- Static files

What is context?

- Data sent from view → template

DJANGO COMMANDS (BASIC)

Must Know:

```
runserver
makemigrations
migrate
createsuperuser
startapp
```

Difference

- **render()** → returns HTML page
- **redirect()** → redirects to another URL

7. DJANGO TEMPLATES TAGS (BASIC IDEA)

Example:

```
{% for item in data %}
{{ item.name }}
{% endfor %}
```

☐ Interview Answer

Django template tags are used to write logic inside HTML like loops and conditions.

. FOREIGN KEY (DATABASE RELATION)

☐ Example:

```
class Course(models.Model):
    name = models.CharField(max_length=100)

class Student(models.Model):
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
```

☐ Interview Answer

ForeignKey is used to create relationships between tables in Django models.